Understanding Parse Trees and Grammar Verification

A Step-by-Step Guide to Verifying Strings in Formal Languages

What is a Grammar?

A formal grammar is a set of production rules that describe how to form valid strings in a language. Understanding these components is essential for constructing parse trees and verifying string membership.

Terminals

The actual characters or symbols that appear in strings (e.g., A, B, C, x, y, z, comma)

Non-terminals

Variables that represent patterns or structures (e.g., <list>, <item>, <left>, <right>)

Production Rules

Rules that show how non-terminals can be replaced or expanded

Start Symbol

The Grammar We're Working With

This grammar defines lists of nested bracket-like items, where left and right symbols can nest within each other to create structured patterns.

- <list>

  Either a single item, or an item followed by comma and another list (recursive)

- <item>

  Nested structures with matching left and right symbols, like balanced brackets

- <left>

  Can be A, B, or C (opening symbols)

- <right>

  Can be x, y, or z (closing symbols)

<list> → <item> , <list> | <item>

<item> → <left> <item> <right> | <left> <right>

<left> → A | B | C

<right> → x | y | z

Example strings: "Ax", "BAy", "Ax,Cy", "AByx,Cz"

What is a Parse Tree?

A parse tree is a hierarchical diagram that shows how a string can be derived from a grammar's start symbol by applying production rules step by step.

Root

The start symbol at the top of the tree (e.g., <list>) where all derivations begin.

Internal Nodes

Non-terminal symbols that represent intermediate steps in the derivation process.

Leaves

Terminal symbols that form the actual string when read from left to right.

Edges

Connections that represent the application of specific production rules.

```
<list>
  |
+-----+-----+
|     |     |
<item> , <list>
  |        |
+--+--+   <item>
|     |     |
<left> <right> ...
  |     |
  A     x
```

If a valid parse tree exists, the string belongs to the language.

Choosing a Valid String

Ax,By

"Ax" is a simple item: A (left) followed by x (right)

"By" is another simple item: B (left) followed by y (right)

They are connected by a comma

This matches the pattern: <item> , <list> where the second part is just another <item>

We will construct a complete parse tree to prove this claim in the next slide.

Building the Parse Tree

We construct the parse tree from the top down, applying production rules step by step to derive "Ax,By".

1. Start with <list>
2. Apply rule: <list> → <item> , <list>
3. First <item> → <left> <right> → A x
4. Second <list> → <item> → <left> <right> → B y
5. ✓ Leaves read left-to-right: A x , B y

```
                <list>
                  |
      +-----------+-----------+
      |         |         |
    <item>      ,       <list>
      |                   |
  +----+----+            <item>
  |       |              |
<left>  <right>        +----+----+
  |       |          |        |
  A       x        <left>  <right>
                     |        |
                     B        y
```

The Parse Tree Confirms Validity

The complete parse tree demonstrates that "Ax,By" belongs to the language. Each step follows a valid production rule, and the result matches our target string exactly.

Valid Rules

Every step follows a valid production rule from the grammar with no invented or modified rules

Correct Root

The root is the start symbol <list>, which is where all derivations must begin

Exact Match

The leaves produce exactly "Ax,By" when read left-to-right with no extra or missing characters

No Violations

Key Insight

The ability to construct this parse tree is proof that "Ax,By" belongs to the language. This is the fundamental verification technique for formal languages.

Identifying Invalid Strings

The string "xA" uses valid terminal symbols but violates the grammar's structural requirements.

Problem

The grammar requires <left> symbols (A, B, C) to appear before <right> symbols (x, y, z) in every <item>

Production Rules

<item> → <left> <item> <right> | <left> <right>

(left always comes first)

Conclusion

Even though "xA" uses valid terminal symbols, it cannot be generated by any sequence of production rules

xA

Why "xA" is Invalid

Invalid strings cannot be derived using the grammar's rules because they violate structural requirements.

Problem

The grammar requires <left> symbols (A, B, C) to appear before <right> symbols (x, y, z) in every <item>

Production Rules

<item> → <left> <item> <right> (left comes first)

<item> → <left> <right> (left comes first)

No Rule Allows

<right> to appear before <left>, or standalone terminals without proper structure

Conclusion

"xA" cannot be generated by any sequence of production rules, so it does NOT belong to the language

xA

✗ <right> before <left>

✗ No valid derivation

✗ Violates grammar structure

Key Takeaways

Summary of Key Concepts

1. Grammars define languages through production rules that specify valid string structures
2. Parse trees provide visual proof that a string can be generated by a grammar
3. Valid strings can be fully derived from the start symbol using only the grammar's rules
4. Invalid strings cannot be parsed because they violate the grammar's structural requirements
5. Terminal symbols alone don't guarantee validity — the structure and ordering matter

Practice Strategy

- Start with simple strings and build parse trees step-by-step
- Check each production rule application carefully
- For invalid strings, identify which rule is violated
- Use parse trees as both verification tools and learning aids

Next Steps

Practice with increasingly complex grammars and strings to build fluency in formal language theory. Try creating your own examples and challenging yourself to identify both valid and invalid strings.